sloppy

Universidad del Valle de Guatemala
CC3066 – Data Science · Semestre II, 2026

Laboratorio 11: MLOps aplicado al Observatorio de Inteligencia Turística

Informe técnico final – Tikal y Flores, Petén

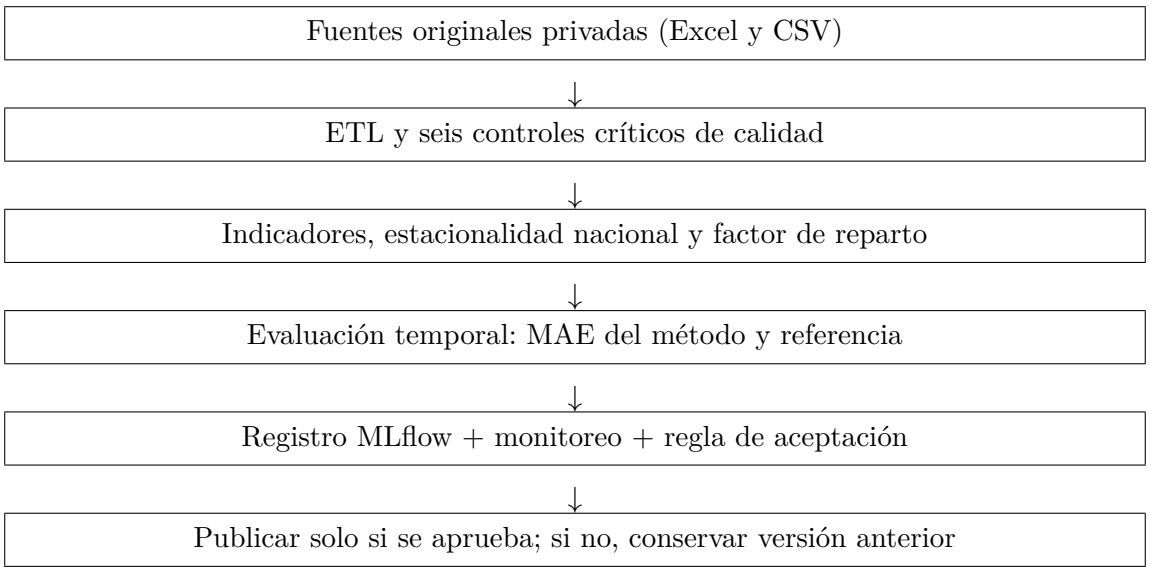
Estudiante: Osman de León
Repositorio: `https://github.com/Emadlgg/proyecto3_Data.git`
Fecha: 2 de noviembre de 2026

1. Objetivo y alcance

El laboratorio integra un flujo reproducible de preparación de datos, validación, estimación, evaluación y publicación controlada para el observatorio turístico de Petén. El procedimiento estima **visitantes por subdestino, segmento y mes** para enero y febrero de 2026, períodos sin cobertura observada de telefonía. Las estimaciones no representan un total único de visitantes del departamento.

Se reutilizaron los módulos existentes `etl.py`, `validaciones.py`, `indicadores.py` y `predicciones.py`. La serie mensual de movilidad y la migración nacional sustentan el cálculo del factor de reparto y del perfil estacional; los datos de Tikal, aeropuerto y hotelería complementan el observatorio, pero no se suman como si midieran el mismo universo. El ajuste del factor utiliza septiembre–diciembre de 2025; la evaluación temporal utiliza meses observados anteriores al objetivo y evita incorporar el mes evaluado al cálculo de sus parámetros.

2. Arquitectura del flujo



Streamlit consulta únicamente versiones aprobadas

El flujo se ejecuta mediante `python -m src.pipeline -config config/mlops.json`. Cada ejecución conserva un identificador, huellas SHA-256 de fuentes y salidas, versión de configuración, commit de Git, métricas y registro JSON. El tablero no vuelve a estimar al cambiar filtros.

3. Controles de calidad y monitoreo

Se implementaron controles de columnas y tipos, fechas y cobertura, unicidad de claves, faltantes críticos, valores no finitos o negativos y reglas metodológicas. En particular, no se sustituyen meses sin cobertura por cero, no se suman presencias diarias para inferir visitantes mensuales y no se agregan subdestinos para producir visitantes únicos de Petén. Un fallo crítico impide publicar.

El monitoreo revisa actualización, calidad y valores anómalos de las estimaciones. La ejecución local reportada produjo **cero alertas**; esto no demuestra por sí solo precisión predictiva. Persisten dos advertencias documentadas: siete meses con cero en Tikal cuyo significado requiere confirmación y el tratamiento de la oferta hotelera como inventario por corte.

4. Evaluación y decisión

La evaluación retrospectiva compara el procedimiento estacional con una referencia sencilla basada en el último valor observado, sobre los mismos meses. El error absoluto medio se calcula como

$$\mathrm{MAE} = \frac{1}{n}\sum_{t=1}^{n} |y_t - \widehat{y}_t|.$$

Indicador	**Resultado**
MAE del procedimiento	9 469.14
MAE de la referencia	5 079.95
Razón MAE (procedimiento / referencia)	1.8640
nMAE del procedimiento	0.1724 (17.24 %)
Umbral máximo de razón MAE	1.00
Umbral máximo de nMAE	0.35

El nMAE cumple su límite, pero la razón MAE excede 1.00: el procedimiento presenta aproximadamente 86.4 % más error absoluto medio que la referencia. Por tanto, la decisión registrada fue `CANDIDATO_RECHAZADO_NO_PUBLICADO`. No se cambiaron los umbrales después de conocer el resultado ni se calculó error para enero y febrero de 2026 sin observaciones comparables.

5. Trazabilidad, reproducibilidad y Docker

MLflow registró métricas, parámetros, huellas de las fuentes y la etiqueta del commit `82d74a1e` (abreviado). Se realizaron dos ejecuciones con iguales datos y configuración: coincidieron los hashes de configuración y fuentes, la decisión y los SHA-256 de todas las salidas. Esto respalda la reproducibilidad exacta de los archivos generados en esa comprobación.

El despliegue local con Docker Compose levantó **Streamlit** en `localhost:8501` y **MLflow** en `localhost:5000`. El pipeline se ejecutó dentro de un contenedor y registró la ejecución en MLflow. Se corrigió la validación del encabezado Host para la comunicación interna. Tras recrear los contenedores sin eliminar el volumen, se comprobó que una ejecución anterior seguía disponible con métricas, parámetros y commit: evidencia de persistencia del almacenamiento.

6. Fallo controlado y recuperación

En pruebas automatizadas se verificó el rechazo de columnas ausentes, fechas inválidas, claves duplicadas, faltantes críticos, conteos negativos y valores infinitos. Las **78 pruebas** de la suite completa pasaron en la ejecución registrada. Adicionalmente, **3 pruebas de integración** comprobaron que un candidato rechazado no crea una versión publicada, que un fallo no sustituye la versión anterior y que una versión corrupta no se muestra.

La demostración de publicación y restauración se realizó con **datos sintéticos aislados**. Tras crear dos versiones, rechazar una actualización y restaurar la primera, la versión vigente siguió siendo `DEMO_SINTETICA_1`, con la misma huella SHA-256. Esta evidencia no equivale a ejecutar el pipeline completo con una copia defectuosa de una fuente institucional; esa prueba integral queda como limitación pendiente. Ninguna predicción real rechazada fue marcada como aprobada.

7. Conclusiones y limitaciones

El observatorio dispone de un pipeline reproducible, controles de calidad, evaluación temporal, registro en MLflow, despliegue Docker y mecanismos de publicación y recuperación probados de forma aislada. El sistema aplica correctamente una regla de aceptación que, en los datos evaluados, **rechaza** el procedimiento frente a la referencia. En consecuencia, el tablero debe indicar que no hay estimaciones reales aprobadas mientras no exista una versión que cumpla los criterios.

Los registros y datos institucionales se mantienen en almacenamiento privado y no deben publicarse en GitHub ni incorporarse a imágenes Docker. Como trabajo pendiente queda demostrar el fallo integral con una copia alterada de las fuentes y documentar esa ejecución en MLflow. La decisión de rechazo actual se conserva como resultado válido del laboratorio.